

Composición de Componentes

Objetivos

- Entender por qué React favorece composición sobre herencia
- Aplicar patrones de composición en componentes
- Crear componentes genéricos y reutilizables
- Usar Higher-Order Components (HOCs) cuando sea apropiado

Contenido

1. ¿Por qué Composición en React?

React adopta un modelo de **composición** en lugar de herencia de clases. Esto significa que construimos componentes combinando otros componentes, no extendiendo clases base.

```
// ❌ NO en React - Herencia (evitar)
class Button extends BaseComponent {
  render() {
    return <button>{this.props.text}</button>;
  }
}

// ✅ SÍ en React - Composición
// QUÉ: Componente base que acepta children y variant como props
// PARA: Crear un botón flexible que puede contener cualquier contenido
// IMPACTO: Un solo componente sirve para múltiples casos de uso
const Button: React.FC<ButtonProps> = ({ children, variant }) => {
  return <button className={`btn btn-${variant}`}>{children}</button>;
};

// QUÉ: Componentes especializados que envuelven al componente base
// PARA: Crear variantes pre-configuradas sin duplicar código
// IMPACTO: API simple para el usuario, mantenimiento centralizado
const PrimaryButton: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => <Button variant="primary">{children}</Button>;

const DangerButton: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => <Button variant="danger">{children}</Button>;
```

2. Beneficios de la Composición

 Diagrama de Composición

2.1 Flexibilidad

La composición permite combinar componentes de formas que no anticipamos:

```
// QUÉ: Interface que define props con children genérico
// PARA: Permitir que Card acepte cualquier contenido JSX
// IMPACTO: Máxima flexibilidad - el padre decide qué renderizar
```

```

interface CardProps {
  children: React.ReactNode;
  className?: string;
}

// QUÉ: Componente contenedor que solo proporciona estructura visual
// PARA: Encapsular estilos sin imponer estructura de contenido
// IMPACTO: Reutilizable en contextos completamente diferentes
const Card: React.FC<CardProps> = ({ children, className = '' }) => (
  <div className={`card ${className}`}>{children}</div>
);

// Múltiples formas de usarlo
const UserProfile: React.FC<{ user: User }> = ({ user }) => (
  <Card>
    <img
      src={user.avatar}
      alt={user.name}
    />
    <h2>{user.name}</h2>
    <p>{user.bio}</p>
  </Card>
);

const ProductCard: React.FC<{ product: Product }> = ({ product }) => (
  <Card className="product-card">
    <img
      src={product.image}
      alt={product.name}
    />
    <h3>{product.name}</h3>
    <span className="price">${product.price}</span>
  </Card>
);

const NotificationCard: React.FC<{ message: string }> = ({ message }) => (
  <Card className="notification">
    <p>{message}</p>
  </Card>
);

```

2.2 Reutilización

Componentes pequeños y enfocados son más fáciles de reutilizar:

```

// QUÉ: Componentes atómicos - la unidad más pequeña de UI
// PARA: Crear piezas reutilizables que hacen una sola cosa bien
// IMPACTO: Se pueden combinar de infinitas formas sin modificar el código
const Avatar: React.FC<{
  src: string;
  alt: string;
  size?: 'sm' | 'md' | 'lg';
}> = ({ src, alt, size = 'md' }) => (
  <img
    src={src}
    alt={alt}

```

```

    className={`avatar avatar-${size}`}
  />
);

const Badge: React.FC<{ children: React.ReactNode; color?: string }> = ({
  children,
  color = 'blue',
}) => <span className={`badge badge-${color}`}>{children}</span>;

const Text: React.FC<{
  children: React.ReactNode;
  variant?: 'body' | 'caption';
}> = ({ children, variant = 'body' }) => (
  <p className={`text-${variant}`}>{children}</p>
);

// QUÉ: Componente compuesto que usa los atómicos
// PARA: Crear UI más compleja combinando piezas simples
// IMPACTO: Código declarativo, fácil de leer y mantener
const UserBadge: React.FC<{ user: User }> = ({ user }) => (
  <div className="user-badge">
    <Avatar
      src={user.avatar}
      alt={user.name}
      size="sm"
    />
    <Text variant="body">{user.name}</Text>
    {user.isAdmin && <Badge color="gold">Admin</Badge>}
  </div>
);

```

2.3 Testabilidad

Componentes compuestos son más fáciles de testear:

```

// Fácil de testear individualmente
describe('Avatar', () => {
  it('renders with correct size class', () => {
    render(
      <Avatar
        src="/test.jpg"
        alt="Test"
        size="lg"
      />,
    );
    expect(screen.getByRole('img')).toHaveClass('avatar-lg');
  });
});

describe('Badge', () => {
  it('renders children correctly', () => {
    render(<Badge color="green">Active</Badge>);
    expect(screen.getByText('Active')).toBeInTheDocument();
  });
});

```

```
// Fácil de testear la composición
describe('UserBadge', () => {
  it('shows admin badge for admin users', () => {
    const adminUser = { name: 'Admin', avatar: '/admin.jpg', isAdmin: true };
    render(<UserBadge user={adminUser} />);
    expect(screen.getByText('Admin')).toBeInTheDocument();
  });
});
```

3. Patrones de Composición

3.1 Contenedor y Presentación

Separar lógica de presentación:

```
// Componente de presentación - solo UI
interface UserListViewProps {
  users: User[];
  onSelectUser: (user: User) => void;
  selectedId?: string;
}

const UserListView: React.FC<UserListViewProps> = ({
  users,
  onSelectUser,
  selectedId,
}) => (
  <ul className="user-list">
    {users.map((user) => (
      <li
        key={user.id}
        className={user.id === selectedId ? 'selected' : ''}
        onClick={() => onSelectUser(user)}>
        <Avatar
          src={user.avatar}
          alt={user.name}
          size="sm"
        />
        <span>{user.name}</span>
      </li>
    ))}
  </ul>
);

// Componente contenedor - maneja lógica y estado
const UserListContainer: React.FC = () => {
  const [users, setUsers] = useState<User[]>([]);
  const [selectedUser, setSelectedUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchUsers()
      .then(setUsers)
      .finally(() => setLoading(false));
  }, []);
```

```

if (loading) return <Spinner />;

return (
  <UserListView
    users={users}
    onSelectUser={setSelectedUser}
    selectedId={selectedUser?.id}
  />
);
};

```

3.2 Especialización

Crear versiones especializadas de componentes genéricos:

```

// Componente genérico
interface DialogProps {
  title: string;
  children: React.ReactNode;
  onClose: () => void;
  actions?: React.ReactNode;
}

const Dialog: React.FC<DialogProps> = ({
  title,
  children,
  onClose,
  actions,
}) => (
  <div
    className="dialog-overlay"
    onClick={onClose}>
    <div
      className="dialog"
      onClick={(e) => e.stopPropagation()}>
      <header className="dialog-header">
        <h2>{title}</h2>
        <button
          onClick={onClose}
          aria-label="Cerrar">
          x
        </button>
      </header>
      <div className="dialog-body">{children}</div>
      {actions && <footer className="dialog-actions">{actions}</footer>}
    </div>
  </div>
);

// Especialización: Diálogo de confirmación
interface ConfirmDialogProps {
  title: string;
  message: string;
  onConfirm: () => void;
  onCancel: () => void;
  confirmText?: string;
}

```

```

    cancelText?: string;
  }

const ConfirmDialog: React.FC<ConfirmDialogProps> = ({
  title,
  message,
  onConfirm,
  onCancel,
  confirmText = 'Confirmar',
  cancelText = 'Cancelar',
}) => (
  <Dialog
    title={title}
    onClose={onCancel}
    actions={
      <>
        <button
          onClick={onCancel}
          className="btn-secondary">
            {cancelText}
          </button>
        <button
          onClick={onConfirm}
          className="btn-primary">
            {confirmText}
          </button>
        </>
      </>
    }
  >
    <p>{message}</p>
  </Dialog>
);

// Especialización: Diálogo de error
const ErrorDialog: React.FC<{ error: string; onClose: () => void }> = ({
  error,
  onClose,
}) => (
  <Dialog
    title="Error"
    onClose={onClose}
    actions={
      <button
        onClick={onClose}
        className="btn-primary">
          Entendido
        </button>
      >
    >
    <div className="error-content">
      <span className="error-icon">⚠️</span>
      <p>{error}</p>
    </div>
  </Dialog>
);

```

3.3 Inversión de Control

Dejar que el padre controle el comportamiento:

```
// Componente con inversión de control
interface ListProps<T> {
  items: T[];
  renderItem: (item: T, index: number) => React.ReactNode;
  keyExtractor: (item: T) => string;
  emptyMessage?: string;
}

function List<T>({
  items,
  renderItem,
  keyExtractor,
  emptyMessage,
}: ListProps<T>) {
  if (items.length === 0) {
    return <p className="empty-list">{emptyMessage ?? 'No hay elementos'}</p>;
  }

  return (
    <ul className="list">
      {items.map((item, index) => (
        <li key={keyExtractor(item)}>{renderItem(item, index)}</li>
      ))}
    </ul>
  );
}

// Uso - el padre controla cómo se renderiza cada item
const UserDirectory: React.FC = () => {
  const users: User[] = [
    /* ... */
  ];

  return (
    <List
      items={users}
      keyExtractor={(user) => user.id}
      renderItem={(user) => (
        <div className="user-row">
          <Avatar
            src={user.avatar}
            alt={user.name}
          />
          <span>{user.name}</span>
          <span className="email">{user.email}</span>
        </div>
      )}
      emptyMessage="No hay usuarios registrados"
    />
  );
};

const ProductCatalog: React.FC = () => {
  const products: Product[] = [
```

```

    /* ... */
  ];

  return (
    <List
      items={products}
      keyExtractor={(product) => product.sku}
      renderItem={(product) => (
        <div className="product-item">
          <img
            src={product.image}
            alt={product.name}
          />
          <h3>{product.name}</h3>
          <span className="price">${product.price}</span>
        </div>
      )}
    />
  );
};

```

4. Higher-Order Components (HOCs)

Los HOCs son funciones que reciben un componente y retornan uno nuevo con funcionalidad adicional:

```

// HOC para añadir loading state
function withLoading<P extends object>({
  WrappedComponent: React.ComponentType<P>,
  loadingMessage: string = 'Cargando...',
}) {
  interface WithLoadingProps {
    isLoading: boolean;
  }

  return function WithLoadingComponent(props: P & WithLoadingProps) {
    const { isLoading, ...componentProps } = props;

    if (isLoading) {
      return <div className="loading">{loadingMessage}</div>;
    }

    return <WrappedComponent {...(componentProps as P)} />;
  };
}

// Uso
const UserList: React.FC<{ users: User[] }> = ({ users }) => (
  <ul>
    {users.map((u) => (
      <li key={u.id}>{u.name}</li>
    ))}
  </ul>
);

const UserListWithLoading = withLoading(UserList, 'Cargando usuarios...');

```

```
// En otro componente
const App: React.FC = () => {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);

  return (
    <UserListWithLoading
      users={users}
      isLoading={loading}
    />
  );
};
```

HOC con TypeScript - Mejor tipado

```
// HOC para añadir autenticación
interface WithAuthProps {
  isAuthenticated: boolean;
  user: User | null;
}

function withAuth<P extends object>(WrappedComponent: React.ComponentType<P>) {
  return function WithAuthComponent(props: Omit<P, keyof WithAuthProps>) {
    // Simular contexto de autenticación
    const auth = useAuth();

    if (!auth.isAuthenticated) {
      return <Navigate to="/login" />;
    }

    return (
      <WrappedComponent
        {...(props as P)}
        isAuthenticated={auth.isAuthenticated}
        user={auth.user}
      />
    );
  };
}

// Componente que requiere autenticación
interface DashboardProps extends WithAuthProps {
  title: string;
}

const Dashboard: React.FC<DashboardProps> = ({ title, user }) => (
  <div>
    <h1>{title}</h1>
    <p>Bienvenido, {user?.name}</p>
  </div>
);

const ProtectedDashboard = withAuth(Dashboard);
```

```
// Uso
<ProtectedDashboard title="Mi Dashboard" />;
```

⚠️ **Nota:** Los HOCs fueron populares antes de los hooks. Hoy en día, los **custom hooks** suelen ser preferibles por su simplicidad y mejor composición.

5. Composición vs Herencia - Comparativa

Aspecto	Composición	Herencia
Flexibilidad	Alta - combina componentes libremente	Baja - jerarquía fija
Acoplamiento	Bajo - componentes independientes	Alto - depende de la clase base
Reutilización	Fácil - mezcla y combina	Difícil - hereda todo o nada
Testing	Simple - testea cada pieza	Complejo - necesita mocks de la base
En React	✅ Recomendado	❌ Evitar

6. Cuándo Usar Cada Patrón

```
// ✅ Composición simple con children
// Cuando: el contenido interno es completamente variable
const Card: React.FC<{ children: React.ReactNode }> = ({ children }) => (
  <div className="card">{children}</div>
);

// ✅ Especialización
// Cuando: necesitas versiones pre-configuradas
const SuccessCard = ({ message }: { message: string }) => (
  <Card>
    <span className="icon">✓</span>
    <p>{message}</p>
  </Card>
);

// ✅ Render props / Inversión de control
// Cuando: el padre necesita controlar el renderizado
const DataFetcher = <T,>({
  url,
  render,
}): {
  url: string;
  render: (data: T) => React.ReactNode;
} => {
  const [data, setData] = useState<T | null>(null);
  useEffect(() => {
    fetch(url)
      .then((r) => r.json())
      .then(setData);
  }, [url]);
  return data ? render(data) : <p>Cargando...</p>;
};

// ✅ HOC
```

```
// Cuando: necesitas añadir funcionalidad cross-cutting a múltiples componentes
// (autenticación, logging, analytics)
const withAnalytics = (Component: React.ComponentType) => {
  return (props: any) => {
    useEffect(() => {
      trackPageView(Component.name);
    }, []);
    return <Component {...props} />;
  };
};
```

✓ Checklist de Verificación

- ☐ Entiendo por qué React prefiere composición sobre herencia
- ☐ Puedo crear componentes genéricos que aceptan children
- ☐ Sé cuándo usar especialización vs render props
- ☐ Entiendo los HOCs y cuándo podrían ser útiles
- ☐ Puedo separar componentes de presentación y contenedor

🔗 Recursos Adicionales

- [React Docs - Composition vs Inheritance](#)
- [Patterns.dev - React Patterns](#)
- [Kent C. Dodds - Inversion of Control](#)

Siguiente: [Children y Slots](#)

📖 Children y Slots Pattern

🎯 Objetivos

- Dominar el prop `children` y sus tipos en TypeScript
- Diferenciar entre `ReactNode`, `ReactElement` y otros tipos
- Implementar el patrón Slots para mayor flexibilidad
- Crear componentes con múltiples áreas de contenido

📋 Contenido

1. El Prop Children

En React, `children` es un prop especial que permite pasar elementos JSX entre las etiquetas de apertura y cierre de un componente:

```
// children es lo que está entre las etiquetas
<Card>
  <h2>Título</h2>
  <p>Contenido del card</p>
</Card>
```

 Diagrama de Children

1.1 Tipando Children

```
import { ReactNode, ReactElement, PropsWithChildren } from 'react';

// QUÉ: ReactNode es el tipo más amplio para children
// PARA: Aceptar cualquier contenido renderizable (elementos, strings, numbers, null)
// IMPACTO: Máxima flexibilidad - el componente funciona con cualquier contenido
interface CardProps {
  children: ReactNode;
  title?: string;
}

// QUÉ: Componente que renderiza children dentro de su estructura
// PARA: Crear contenedores que encapsulan estilos sin imponer contenido
// IMPACTO: El padre decide qué va dentro, el componente decide cómo se ve
const Card: React.FC<CardProps> = ({ children, title }) => (
  <div className="card">
    {title && <h2>{title}</h2>}
    <div className="card-content">{children}</div>
  </div>
);

// Todos estos usos son válidos con ReactNode
<Card>
  <p>Texto en un elemento</p>
</Card>

<Card>Texto directo</Card>

<Card>{42}</Card>

<Card>{null}</Card>

<Card>
  <>
    <p>Fragmento</p>
    <p>Con múltiples hijos</p>
  </>
</Card>
```

1.2 Tipos de Children

```
import {
  ReactNode,
  ReactElement,
  ReactChild,
  JSX,
} from 'react';

// QUÉ: ReactNode - tipo que acepta cualquier cosa renderizable
// PARA: Máxima flexibilidad cuando no importa qué tipo de contenido se pasa
// IMPACTO: Acepta strings, numbers, elementos, null, arrays, fragments
type ReactNode = /* tipo amplio */;

// QUÉ: ReactElement - tipo que solo acepta elementos JSX
```

```

// PARA: Forzar que se pase un componente, no texto plano
// IMPACTO: TypeScript rechaza strings o numbers, solo acepta <Component />
type ReactElement = /* elemento JSX específico */;

// Ejemplos de uso según necesidad

// 1. Aceptar cualquier contenido renderizable
interface FlexibleContainerProps {
  children: ReactNode;
}

// 2. Requerir exactamente un elemento JSX
interface SingleChildProps {
  children: ReactElement;
}

// QUÉ: Props que mezclan ReactElement y ReactNode según necesidad
// PARA: Forzar que icon sea un componente pero children puede ser texto
// IMPACTO: API más segura - previene errores como pasar emoji como icon
interface IconButtonProps {
  icon: ReactElement; // Debe ser un elemento, no texto
  children: ReactNode; // Texto del botón puede ser string
}

const IconButton: React.FC<IconButtonProps> = ({ icon, children }) => (
  <button className="icon-button">
    <span className="icon">{icon}</span>
    <span className="text">{children}</span>
  </button>
);

// Uso correcto
<IconButton icon={<SearchIcon />}>Buscar</IconButton>

// ❌ Error de TypeScript - string no es ReactElement
<IconButton icon="🔍">Buscar</IconButton>

```

1.3 PropsWithChildren

TypeScript ofrece un helper para añadir children a tus props:

```

import { PropsWithChildren } from 'react';

// Sin PropsWithChildren
interface CardProps {
  title: string;
  children: ReactNode;
}

// QUÉ: PropsWithChildren es un utility type de React
// PARA: Añadir automáticamente children?: ReactNode a cualquier tipo
// IMPACTO: Menos código repetitivo, children siempre es opcional
type CardProps = PropsWithChildren<{
  title: string;
}>;

```

```
// Ambos resultan en:
// { title: string; children?: ReactNode }

const Card: React.FC<CardProps> = ({ title, children }) => (
  <div className="card">
    <h2>{title}</h2>
    {children}
  </div>
);
```

2. Manipulando Children

React provee utilidades para trabajar con children de forma segura:

```
import {
  Children,
  cloneElement,
  isValidElement,
  ReactNode,
  ReactElement,
} from 'react';

interface ListProps {
  children: ReactNode;
}

const List: React.FC<ListProps> = ({ children }) => {
  // Contar children
  const count = Children.count(children);
  console.log(`Lista tiene ${count} elementos`);

  // Iterar sobre children
  const items = Children.map(children, (child, index) => {
    // Verificar si es un elemento válido antes de clonar
    if (isValidElement(child)) {
      // Clonar y añadir props
      return cloneElement(child, {
        className: `list-item item-${index}`,
        'data-index': index,
      });
    }
    return child;
  });

  return <ul className="list">{items}</ul>;
};

// Convertir a array
const childArray = Children.toArray(children);

// Verificar que solo hay un child
const onlyChild = Children.only(children); // Lanza error si hay más de uno
```

Ejemplo práctico: Tabs

```

interface TabsProps {
  children: ReactNode;
  defaultTab?: number;
}

interface TabProps {
  label: string;
  children: ReactNode;
}

const Tab: React.FC<TabProps> = ({ children }) => <div>{children}</div>;

const Tabs: React.FC<TabsProps> = ({ children, defaultTab = 0 }) => {
  const [activeTab, setActiveTab] = useState(defaultTab);

  // Obtener array de children válidos
  const tabs = Children.toArray(children).filter(
    (child): child is ReactElement<TabProps> =>
      isValidElement(child) && child.type === Tab,
  );

  return (
    <div className="tabs">
      {/* Botones de navegación */}
      <div className="tab-buttons">
        {tabs.map((tab, index) => (
          <button
            key={index}
            className={index === activeTab ? 'active' : ''}
            onClick={() => setActiveTab(index)}>
            {tab.props.label}
          </button>
        ))}
      </div>

      {/* Contenido del tab activo */}
      <div className="tab-content">{tabs[activeTab]}</div>
    </div>
  );
};

// Uso
<Tabs defaultTab={0}>
  <Tab label="Perfil">
    <ProfileContent />
  </Tab>
  <Tab label="Configuración">
    <SettingsContent />
  </Tab>
  <Tab label="Notificaciones">
    <NotificationsContent />
  </Tab>
</Tabs>;

```

3. Slots Pattern

El patrón Slots permite definir **áreas específicas** donde el padre puede inyectar contenido:

 Diagrama de Slots

3.1 Slots con Props Explícitas

```
// QUÉ: Interface con múltiples props que actúan como "slots" de contenido
// PARA: Definir áreas específicas donde el padre puede inyectar contenido
// IMPACTO: Estructura predecible con flexibilidad en cada zona
interface CardProps {
  // Slots definidos como props
  header?: ReactNode;
  footer?: ReactNode;
  children: ReactNode; // Slot principal (body)
}

// QUÉ: Componente con múltiples zonas de contenido
// PARA: El padre controla qué va en cada zona sin modificar el componente
// IMPACTO: Mismo componente sirve para cards muy diferentes
const Card: React.FC<CardProps> = ({ header, footer, children }) => (
  <div className="card">
    {header && <header className="card-header">{header}</header>}
    <main className="card-body">{children}</main>
    {footer && <footer className="card-footer">{footer}</footer>}
  </div>
);

// Uso
<Card
  header=<h2>Título del Card</h2>
  footer={
    <div className="actions">
      <button>Cancelar</button>
      <button>Guardar</button>
    </div>
  }>
  <p>Este es el contenido principal del card.</p>
  <p>Puede tener múltiples elementos.</p>
</Card>;
```

3.2 Slots Tipados Estrictamente

```
// QUÉ: Interface con tipos específicos para cada slot
// PARA: Forzar que trigger sea un elemento con onClick
// IMPACTO: TypeScript previene errores - no puedes pasar texto como trigger
interface ModalSlots {
  trigger: ReactElement<{ onClick: () => void }>; // Debe tener onClick
  title: ReactNode;
  content: ReactNode;
  actions?: ReactNode;
}
```

```

// QUÉ: Modal que inyecta comportamiento al trigger
// PARA: El padre define el botón, Modal añade la funcionalidad de abrir
// IMPACTO: Separación de responsabilidades - UI vs comportamiento
const Modal: React.FC<ModalSlots> = ({ trigger, title, content, actions }) => {
  const [isOpen, setIsOpen] = useState(false);

  // Clonar trigger para añadir onClick
  const triggerWithHandler = cloneElement(trigger, {
    onClick: () => setIsOpen(true),
  });

  return (
    <>
      {triggerWithHandler}

      {isOpen && (
        <div
          className="modal-overlay"
          onClick={() => setIsOpen(false)}>
          <div
            className="modal"
            onClick={(e) => e.stopPropagation()}>
            <header className="modal-header">
              <h2>{title}</h2>
              <button onClick={() => setIsOpen(false)}>x</button>
            </header>
            <main className="modal-content">{content}</main>
            {actions && <footer className="modal-actions">{actions}</footer>}
          </div>
        </div>
      )}
    </>
  );
};

// Uso
<Modal
  trigger=<button>Abrir Modal</button>
  title="Confirmar acción"
  content=<p>¿Estás seguro de que deseas continuar?</p>
  actions={
    <>
      <button>Cancelar</button>
      <button>Confirmar</button>
    </>
  }
/>;

```

3.3 Slots con Render Props

Para mayor control, combina slots con funciones:

```

interface DataTableProps<T> {
  data: T[];
  // Slots como render props
  renderHeader: () => ReactNode;
}

```

```

renderRow: (item: T, index: number) => ReactNode;
renderFooter?: (totalItems: number) => ReactNode;
renderEmpty?: () => ReactNode;
}

function DataTable<T>({
  data,
  renderHeader,
  renderRow,
  renderFooter,
  renderEmpty,
}: DataTableProps<T>) {
  if (data.length === 0 && renderEmpty) {
    return <div className="table-empty">{renderEmpty()}</div>;
  }

  return (
    <table className="data-table">
      <thead>{renderHeader()}</thead>
      <tbody>
        {data.map((item, index) => (
          <tr key={index}>{renderRow(item, index)}</tr>
        ))}
      </tbody>
      {renderFooter && <tfoot>{renderFooter(data.length)}</tfoot>}
    </table>
  );
}

// Use
interface User {
  id: string;
  name: string;
  email: string;
  role: string;
}

const users: User[] = [
  /* ... */
];

<DataTable
  data={users}
  renderHeader={() => (
    <tr>
      <th>Nombre</th>
      <th>Email</th>
      <th>Rol</th>
    </tr>
  )}
  renderRow={(user) => (
    <>
      <td>{user.name}</td>
      <td>{user.email}</td>
      <td>{user.role}</td>
    </>
  )}
/>

```

```

    })
    renderFooter=(({total}) => (
      <tr>
        <td colSpan={3}>Total: {total} usuarios</td>
      </tr>
    ))
    renderEmpty=({}) => <p>No hay usuarios registrados</p>
  />;

```

4. Layout Components con Slots

Componentes de layout son un caso de uso perfecto para slots:

```

// Layout con múltiples slots
interface PageLayoutProps {
  header?: ReactNode;
  sidebar?: ReactNode;
  children: ReactNode; // Contenido principal
  footer?: ReactNode;
}

const PageLayout: React.FC<PageLayoutProps> = ({
  header,
  sidebar,
  children,
  footer,
}) => (
  <div className="page-layout">
    {header && <header className="layout-header">{header}</header>}

    <div className="layout-body">
      {sidebar && <aside className="layout-sidebar">{sidebar}</aside>}
      <main className="layout-content">{children}</main>
    </div>

    {footer && <footer className="layout-footer">{footer}</footer>}
  </div>
);

// Uso en una página
const DashboardPage: React.FC = () => (
  <PageLayout
    header={<Navbar />}
    sidebar={
      <nav>
        <NavLink to="/dashboard">Inicio</NavLink>
        <NavLink to="/users">Usuarios</NavLink>
        <NavLink to="/settings">Configuración</NavLink>
      </nav>
    }
    footer={<p>© 2024 Mi Aplicación</p>}>
    <h1>Dashboard</h1>
    <DashboardWidgets />
  </PageLayout>
);

```

Layout con Slots Condicionales

```
interface ResponsiveLayoutProps {
  mobileHeader?: ReactNode;
  desktopHeader?: ReactNode;
  children: ReactNode;
}

const ResponsiveLayout: React.FC<ResponsiveLayoutProps> = ({
  mobileHeader,
  desktopHeader,
  children,
}) => {
  const [isMobile, setIsMobile] = useState(window.innerWidth < 768);

  useEffect(() => {
    const handleResize = () => setIsMobile(window.innerWidth < 768);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return (
    <div className={`layout ${isMobile ? 'mobile' : 'desktop'}`}>
      <header>{isMobile ? mobileHeader : desktopHeader}</header>
      <main>{children}</main>
    </div>
  );
};

// Uso
<ResponsiveLayout
  mobileHeader={<MobileNavbar />}
  desktopHeader={<DesktopNavbar />}>
  <PageContent />
</ResponsiveLayout>;
```

5. Validación de Children

A veces necesitas validar qué tipo de children se pasan:

```
import { Children, isValidElement, ReactElement, ReactNode } from 'react';

// Componente que solo acepta ciertos tipos de children
interface TabPanelProps {
  children: ReactNode;
}

interface TabItemProps {
  id: string;
  label: string;
  children: ReactNode;
}

// Marcar componente para identificación
const TabItem: React.FC<TabItemProps> & { displayName: string } = ({
```

```

    children,
  }) => <div>{children}</div>;
  TabItem.displayName = 'TabItem';

const TabPanel: React.FC<TabPanelProps> = ({ children }) => {
  // Validar que todos los children sean TabItem
  const validChildren = Children.toArray(children).filter((child) => {
    if (!isValidElement(child)) {
      console.warn('TabPanel: child inválido ignorado');
      return false;
    }

    // Verificar el tipo del componente
    if ((child.type as any).displayName !== 'TabItem') {
      console.warn(
        `TabPanel: solo acepta TabItem como children, recibió ${
          (child.type as any).displayName || 'unknown'
        }`
      );
      return false;
    }
  });

  return true;
}) as ReactElement<TabItemProps>[];

const [activeId, setActiveId] = useState(validChildren[0]?.props.id);

return (
  <div className="tab-panel">
    <div className="tab-buttons">
      {validChildren.map((child) => (
        <button
          key={child.props.id}
          className={child.props.id === activeId ? 'active' : ''}
          onClick={() => setActiveId(child.props.id)}
        >
          {child.props.label}
        </button>
      ))}
    </div>
    <div className="tab-content">
      {validChildren.find((child) => child.props.id === activeId)}
    </div>
  </div>
);
};

// Uso correcto
<TabPanel>
  <TabItem id="profile" label="Perfil">
    <ProfileContent />
  </TabItem>
  <TabItem id="settings" label="Configuración">
    <SettingsContent />
  </TabItem>
</TabPanel>

```

```
// Uso incorrecto - mostrará warning
<TabPanel>
  <TabItem id="profile" label="Perfil">
    <ProfileContent />
  </TabItem>
  <div>Esto no es un TabItem</div> {/* Warning */}
</TabPanel>
```

6. Children como Función (Render Props)

Patrón avanzado donde children es una función:

```
interface MouseTrackerProps {
  children: (position: { x: number; y: number }) => ReactNode;
}

const MouseTracker: React.FC<MouseTrackerProps> = ({ children }) => {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMove = (e: MouseEvent) => {
      setPosition({ x: e.clientX, y: e.clientY });
    };
    window.addEventListener('mousemove', handleMove);
    return () => window.removeEventListener('mousemove', handleMove);
  }, []);

  // children es una función, la llamamos con los datos
  return <>{children(position)}</>;
};

// Uso - el padre decide qué renderizar con los datos
<MouseTracker>
  {({ x, y }) => (
    <div className="mouse-info">
      <p>
        Posición: ({x}, {y})
      </p>
      <div
        className="cursor-follower"
        style={{ left: x, top: y }}
      />
    </div>
  )}
</MouseTracker>;
```

✓ Checklist de Verificación

- ☐ Sé cuándo usar ReactNode vs ReactElement
- ☐ Puedo crear componentes que aceptan children
- ☐ Entiendo el patrón Slots y cuándo usarlo
- ☐ Puedo manipular children con Children.map, Children.toArray
- ☐ Sé crear layouts flexibles con múltiples slots

- ☐ Entiendo children como función (render props)

Recursos Adicionales

- [React TypeScript Cheatsheet - Children](#)
- [React Docs - Passing JSX as children](#)

Siguiente: [Compound Components](#)

Compound Components Pattern

Objetivos

- Entender el patrón Compound Components
- Implementar componentes compuestos con Context interno
- Crear APIs declarativas y flexibles
- Tipar correctamente compound components en TypeScript

Contenido

1. ¿Qué son los Compound Components?

Los **Compound Components** son un conjunto de componentes que trabajan juntos para formar una funcionalidad completa. Comparten estado implícitamente sin que el usuario tenga que pasarlo manualmente.

 Diagrama de Compound Components

Ejemplo: HTML nativo

HTML tiene compound components nativos:

```
<!-- select y option trabajan juntos -->
<select>
  <option value="a">Opción A</option>
  <option value="b">Opción B</option>
</select>

<!-- table, thead, tbody, tr, td trabajan juntos -->
<table>
  <thead>
    <tr>
      <th>Columna 1</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Dato 1</td>
    </tr>
  </tbody>
</table>
```

El estado fluye implícitamente entre ellos. Vamos a crear lo mismo en React.

2. Compound Components Básico

2.1 Sin Context (limitado)

```
// Versión simple usando cloneElement
interface ToggleProps {
  children: ReactNode;
}

interface ToggleButtonProps {
  children: ReactNode;
  on?: boolean;
  toggle?: () => void;
}

interface ToggleContentProps {
  children: ReactNode;
  on?: boolean;
  when?: boolean;
}

const Toggle: React.FC<ToggleProps> & {
  Button: React.FC<ToggleButtonProps>;
  On: React.FC<ToggleContentProps>;
  Off: React.FC<ToggleContentProps>;
} = ({ children }) => {
  const [on, setOn] = useState(false);
  const toggle = () => setOn((prev) => !prev);

  // Clonar children para inyectar props
  const childrenWithProps = Children.map(children, (child) => {
    if (isValidElement(child)) {
      return cloneElement(child, { on, toggle });
    }
    return child;
  });

  return <div className="toggle">{childrenWithProps}</div>;
};

Toggle.Button = ({ children, on, toggle }) => (
  <button
    onClick={toggle}
    className={on ? 'active' : ''}>
    {children}
  </button>
);

Toggle.On = ({ children, on }) => (on ? <>{children}</> : null);

Toggle.Off = ({ children, on }) => (on ? null : <>{children}</>);

// Uso
<Toggle>
  <Toggle.Button>Activar</Toggle.Button>
  <Toggle.On>✅ Está encendido</Toggle.On>
</Toggle>
```

```
<Toggle.Off>✖ Está apagado</Toggle.Off>
</Toggle>;
```

⚠ **Limitación:** `cloneElement` solo funciona con hijos directos. Si envuelves un componente, pierde los props.

3. Compound Components con Context

La forma correcta y escalable es usar **Context interno**:

```
import { createContext, useContext, useState, ReactNode } from 'react';

// =====
// 1. DEFINIR TIPOS
// =====

// QUÉ: Interface del contexto interno del Accordion
// PARA: Definir qué datos y funciones comparten los subcomponentes
// IMPACTO: Los subcomponentes pueden comunicarse sin props explícitos
interface AccordionContextValue {
  openItems: Set<string>;
  toggleItem: (id: string) => void;
  isOpen: (id: string) => boolean;
}

interface AccordionProps {
  children: ReactNode;
  defaultOpen?: string[];
  allowMultiple?: boolean;
}

interface AccordionItemProps {
  children: ReactNode;
  id: string;
}

interface AccordionTriggerProps {
  children: ReactNode;
}

interface AccordionContentProps {
  children: ReactNode;
}

// =====
// 2. CREAR CONTEXTO INTERNO
// =====

// QUÉ: Contexto interno que NO se exporta al usuario
// PARA: Comunicación entre componentes del mismo grupo sin exponer detalles
// IMPACTO: API limpia para el usuario, estado compartido internamente
const AccordionContext = createContext<AccordionContextValue | null>(null);
const AccordionItemContext = createContext<string | null>(null);

// QUÉ: Hooks internos con validación de contexto
// PARA: Garantizar que subcomponentes se usen dentro del componente padre
// IMPACTO: Errores claros si se usa AccordionTrigger fuera de Accordion
```

```

const useAccordionContext = () => {
  const context = useContext(AccordionContext);
  if (!context) {
    throw new Error('Accordion components must be used within an Accordion');
  }
  return context;
};

const useAccordionItemContext = () => {
  const id = useContext(AccordionItemContext);
  if (!id) {
    throw new Error(
      'AccordionTrigger/Content must be used within AccordionItem',
    );
  }
  return id;
};

// =====
// 3. COMPONENTE PRINCIPAL
// =====

// QUÉ: Componente principal con subcomponentes asignados como propiedades
// PARA: Crear API declarativa tipo <Accordion.Item> en lugar de props complejos
// IMPACTO: Código legible, autocompletado de IDE, flexibilidad de estructura
const Accordion: React.FC<AccordionProps> & {
  Item: React.FC<AccordionItemProps>;
  Trigger: React.FC<AccordionTriggerProps>;
  Content: React.FC<AccordionContentProps>;
} = ({ children, defaultOpen = [], allowMultiple = false }) => {
  const [openItems, setOpenItems] = useState<Set<string>>(new Set(defaultOpen));

  const toggleItem = (id: string) => {
    setOpenItems((prev) => {
      const next = new Set(prev);

      if (next.has(id)) {
        next.delete(id);
      } else {
        if (!allowMultiple) {
          next.clear();
        }
        next.add(id);
      }

      return next;
    });
  };

  const isOpen = (id: string) => openItems.has(id);

  return (
    <AccordionContext.Provider value={{ openItems, toggleItem, isOpen }}>
      <div className="accordion">{children}</div>
    </AccordionContext.Provider>
  );
};

```

```

};

// =====
// 4. COMPONENTES HIJOS
// =====

const AccordionItem: React.FC<AccordionItemProps> = ({ children, id }) => {
  const { isOpen } = useAccordionContext();

  return (
    <AccordionItemContext.Provider value={id}>
      <div className={`accordion-item ${isOpen(id) ? 'open' : ''}`>
        {children}
      </div>
    </AccordionItemContext.Provider>
  );
};

const AccordionTrigger: React.FC<AccordionTriggerProps> = ({ children }) => {
  const { toggleItem, isOpen } = useAccordionContext();
  const id = useAccordionItemContext();

  return (
    <button
      className="accordion-trigger"
      onClick={() => toggleItem(id)}
      aria-expanded={isOpen(id)}>
      {children}
      <span className="icon">{isOpen(id) ? '-' : '+'}</span>
    </button>
  );
};

const AccordionContent: React.FC<AccordionContentProps> = ({ children }) => {
  const { isOpen } = useAccordionContext();
  const id = useAccordionItemContext();

  if (!isOpen(id)) return null;

  return <div className="accordion-content">{children}</div>;
};

// Asignar subcomponentes
Accordion.Item = AccordionItem;
Accordion.Trigger = AccordionTrigger;
Accordion.Content = AccordionContent;

// =====
// 5. USO
// =====

const FAQ: React.FC = () => (
  <Accordion
    defaultOpen={['faq-1']}
    allowMultiple={false}>
    <Accordion.Item id="faq-1">

```

```

    <Accordion.Trigger>¿Qué es React?</Accordion.Trigger>
    <Accordion.Content>
      React es una biblioteca de JavaScript para construir interfaces de
      usuario.
    </Accordion.Content>
  </Accordion.Item>

  <Accordion.Item id="faq-2">
    <Accordion.Trigger>¿Qué es TypeScript?</Accordion.Trigger>
    <Accordion.Content>
      TypeScript es un superset de JavaScript que añade tipado estático.
    </Accordion.Content>
  </Accordion.Item>

  <Accordion.Item id="faq-3">
    <Accordion.Trigger>¿Por qué usar ambos?</Accordion.Trigger>
    <Accordion.Content>
      La combinación proporciona una experiencia de desarrollo superior con
      mejor autocompletado, detección de errores y documentación inline.
    </Accordion.Content>
  </Accordion.Item>
</Accordion>
);

```

4. Compound Components Avanzado: Menu

```

import {
  createContext,
  useContext,
  useState,
  useRef,
  useEffect,
  ReactNode,
} from 'react';

// =====
// TIPOS
// =====

interface MenuContextValue {
  isOpen: boolean;
  open: () => void;
  close: () => void;
  toggle: () => void;
  activeIndex: number;
  setActiveIndex: (index: number) => void;
}

// =====
// CONTEXTO
// =====

const MenuContext = createContext<MenuContextValue | null>(null);

const useMenuContext = () => {

```

```

const context = useContext(MenuContext);
if (!context) {
  throw new Error('Menu components must be used within a Menu');
}
return context;
};

// =====
// COMPONENTES
// =====

interface MenuProps {
  children: ReactNode;
}

const Menu: React.FC & {
  Trigger: React.FC<{ children: ReactNode }>;
  List: React.FC<{ children: ReactNode }>;
  Item: React.FC<{
    children: ReactNode;
    onClick?: () => void;
    disabled?: boolean;
  }>;
  Divider: React.FC;
} = ({ children }) => {
  const [isOpen, setIsOpen] = useState(false);
  const [activeIndex, setActiveIndex] = useState(-1);
  const menuRef = useRef<HTMLDivElement>(null);

  const open = () => setIsOpen(true);
  const close = () => {
    setIsOpen(false);
    setActiveIndex(-1);
  };
  const toggle = () => (isOpen ? close() : open());

  // Cerrar al hacer click fuera
  useEffect(() => {
    const handleClickOutside = (e: MouseEvent) => {
      if (menuRef.current && !menuRef.current.contains(e.target as Node)) {
        close();
      }
    };

    if (isOpen) {
      document.addEventListener('mousedown', handleClickOutside);
    }

    return () => document.removeEventListener('mousedown', handleClickOutside);
  }, [isOpen]);

  // Cerrar con Escape
  useEffect(() => {
    const handleEscape = (e: KeyboardEvent) => {
      if (e.key === 'Escape') close();
    };
  });

```

```

    if (isOpen) {
      document.addEventListener('keydown', handleEscape);
    }

    return () => document.removeEventListener('keydown', handleEscape);
  }, [isOpen]);

  return (
    <MenuContext.Provider
      value={{ isOpen, open, close, toggle, activeIndex, setActiveIndex }}>
      <div
        className="menu"
        ref={menuRef}>
        {children}
      </div>
    </MenuContext.Provider>
  );
};

const MenuTrigger: React.FC<{ children: ReactNode }> = ({ children }) => {
  const { toggle, isOpen } = useMenuContext();

  return (
    <button
      className="menu-trigger"
      onClick={toggle}
      aria-expanded={isOpen}
      aria-haspopup="menu">
      {children}
    </button>
  );
};

const MenuList: React.FC<{ children: ReactNode }> = ({ children }) => {
  const { isOpen } = useMenuContext();

  if (!isOpen) return null;

  return (
    <ul
      className="menu-list"
      role="menu">
      {children}
    </ul>
  );
};

const MenuItem: React.FC<{
  children: ReactNode;
  onClick?: () => void;
  disabled?: boolean;
}> = ({ children, onClick, disabled = false }) => {
  const { close } = useMenuContext();

  const handleClick = () => {

```

```

    if (disabled) return;
    onClick?.();
    close();
  };

  return (
    <li
      className={`menu-item ${disabled ? 'disabled' : ''}`}
      role="menuitem"
      onClick={handleClick}
      aria-disabled={disabled}>
      {children}
    </li>
  );
};

const MenuDivider: React.FC = () => (
  <li
    className="menu-divider"
    role="separator"
  />
);

// Asignar subcomponentes
Menu.Trigger = MenuTrigger;
Menu.List = MenuList;
Menu.Item = MenuItem;
Menu.Divider = MenuDivider;

// =====
// USO
// =====

const UserMenu: React.FC = () => (
  <Menu>
    <Menu.Trigger>
      <span>👤 Mi Cuenta</span>
    </Menu.Trigger>
    <Menu.List>
      <Menu.Item onClick={() => console.log('Perfil')}>Ver Perfil</Menu.Item>
      <Menu.Item onClick={() => console.log('Configuración')}>
        Configuración
      </Menu.Item>
      <Menu.Divider />
      <Menu.Item onClick={() => console.log('Cerrar sesión')}>
        Cerrar Sesión
      </Menu.Item>
    </Menu.List>
  </Menu>
);

```

5. Compound Components con Estado Controlado

A veces quieres que el padre pueda controlar el estado:

```

interface TabsProps {
  children: ReactNode;
  // Modo no controlado
  defaultValue?: string;
  // Modo controlado
  value?: string;
  onChange?: (value: string) => void;
}

interface TabsContextValue {
  activeTab: string;
  setActiveTab: (value: string) => void;
}

const TabsContext = createContext<TabsContextValue | null>(null);

const useTabs = () => {
  const context = useContext(TabsContext);
  if (!context) {
    throw new Error('Tabs components must be used within Tabs');
  }
  return context;
};

const Tabs: React.FC<TabsProps> & {
  List: React.FC<{ children: ReactNode }>;
  Tab: React.FC<{ value: string; children: ReactNode }>;
  Panel: React.FC<{ value: string; children: ReactNode }>;
} = ({ children, defaultValue = '', value, onChange }) => {
  // Estado interno para modo no controlado
  const [internalValue, setInternalValue] = useState(defaultValue);

  // Determinar si es controlado o no
  const isControlled = value !== undefined;
  const activeTab = isControlled ? value : internalValue;

  const setActiveTab = (newValue: string) => {
    // Si es controlado, solo notificar al padre
    if (isControlled) {
      onChange?.(newValue);
    } else {
      // Si no es controlado, actualizar estado interno
      setInternalValue(newValue);
      onChange?.(newValue); // También notificar si hay callback
    }
  };

  return (
    <TabsContext.Provider value={{ activeTab, setActiveTab }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
};

const TabsList: React.FC<{ children: ReactNode }> = ({ children }) => (
  <div

```

```

        className="tabs-list"
        role="tablist">
        {children}
    </div>
);

const Tab: React.FC<{ value: string; children: ReactNode }> = ({
    value,
    children,
}) => {
    const { activeTab, setActiveTab } = useTabs();
    const isActive = activeTab === value;

    return (
        <button
            className={`tab ${isActive ? 'active' : ''}`}
            role="tab"
            aria-selected={isActive}
            onClick={() => setActiveTab(value)}>
            {children}
        </button>
    );
};

const TabPanel: React.FC<{ value: string; children: ReactNode }> = ({
    value,
    children,
}) => {
    const { activeTab } = useTabs();

    if (activeTab !== value) return null;

    return (
        <div
            className="tab-panel"
            role="tabpanel">
            {children}
        </div>
    );
};

Tabs.List = TabsList;
Tabs.Tab = Tab;
Tabs.Panel = TabPanel;

// =====
// USO - Modo No Controlado
// =====

const UncontrolledExample: React.FC = () => (
    <Tabs
        defaultValue="tab1"
        onChange={(v) => console.log('Tab changed:', v)}>
        <Tabs.List>
            <Tabs.Tab value="tab1">Tab 1</Tabs.Tab>
            <Tabs.Tab value="tab2">Tab 2</Tabs.Tab>
        </Tabs.List>
    </Tabs>
);

```

```

    </Tabs.List>
    <Tabs.Panel value="tab1">Contenido 1</Tabs.Panel>
    <Tabs.Panel value="tab2">Contenido 2</Tabs.Panel>
  </Tabs>
);

// =====
// USO - Modo Controlado
// =====

const ControlledExample: React.FC = () => {
  const [activeTab, setActiveTab] = useState('tab1');

  return (
    <>
      <p>Tab activo: {activeTab}</p>
      <button onClick={() => setActiveTab('tab1')}>Ir a Tab 1</button>
      <button onClick={() => setActiveTab('tab2')}>Ir a Tab 2</button>

      <Tabs
        value={activeTab}
        onChange={setActiveTab}>
        <Tabs.List>
          <Tabs.Tab value="tab1">Tab 1</Tabs.Tab>
          <Tabs.Tab value="tab2">Tab 2</Tabs.Tab>
        </Tabs.List>
        <Tabs.Panel value="tab1">Contenido 1</Tabs.Panel>
        <Tabs.Panel value="tab2">Contenido 2</Tabs.Panel>
      </Tabs>
    </>
  );
};

```

6. Tipado Estricto con displayName

Para TypeScript más estricto y mejor debugging:

```

// Definir el tipo del compound component completo
interface CardComponent extends React.FC<{
  children: ReactNode;
  className?: string;
}> {
  Header: React.FC<{ children: ReactNode }>;
  Body: React.FC<{ children: ReactNode }>;
  Footer: React.FC<{ children: ReactNode }>;
  Image: React.FC<{ src: string; alt: string }>;
}

// Crear componentes individuales
const CardHeader: React.FC<{ children: ReactNode }> = ({ children }) => (
  <header className="card-header">{children}</header>
);
CardHeader.displayName = 'Card.Header';

const CardBody: React.FC<{ children: ReactNode }> = ({ children }) => (

```

```

    <main className="card-body">{children}</main>
  );
  CardBody.displayName = 'Card.Body';

const CardFooter: React.FC<{ children: ReactNode }> = ({ children }) => (
  <footer className="card-footer">{children}</footer>
);
CardFooter.displayName = 'Card.Footer';

const CardImage: React.FC<{ src: string; alt: string }> = ({ src, alt }) => (
  <img
    className="card-image"
    src={src}
    alt={alt}
  />
);
CardImage.displayName = 'Card.Image';

// Componente principal
const Card: CardComponent = ({ children, className = '' }) => (
  <article className={`card ${className}`}>{children}</article>
);
Card.displayName = 'Card';

// Asignar subcomponentes
Card.Header = CardHeader;
Card.Body = CardBody;
Card.Footer = CardFooter;
Card.Image = CardImage;

// Uso con autocompletado perfecto
<Card className="product-card">
  <Card.Image
    src="/product.jpg"
    alt="Producto"
  />
  <Card.Header>
    <h3>Nombre del Producto</h3>
  </Card.Header>
  <Card.Body>
    <p>Descripción del producto...</p>
  </Card.Body>
  <Card.Footer>
    <button>Comprar</button>
  </Card.Footer>
</Card>;

```

7. Beneficios del Patrón

Beneficio	Descripción
API Declarativa	El código es legible y auto-documentado
Flexibilidad	El usuario decide la estructura
Encapsulación	Estado interno oculto del consumidor

Reutilización	Subcomponentes reutilizables
TypeScript	Autocompletado y type safety

✓ Checklist de Verificación

- ☐ Entiendo qué son los Compound Components
- ☐ Sé crear contexto interno para compartir estado
- ☐ Puedo implementar hooks seguros que validan contexto
- ☐ Entiendo la diferencia entre modo controlado y no controlado
- ☐ Sé usar displayName para mejor debugging
- ☐ Puedo crear APIs declarativas y flexibles

Recursos Adicionales

- [Kent C. Dodds - Compound Components](#)
- [Patterns.dev - Compound Pattern](#)
- [Headless UI](#) - Ejemplos de compound components

Siguiente: [Context API](#)

Context API con TypeScript

Objetivos

- Entender cuándo usar Context vs props
- Crear contextos tipados correctamente
- Implementar providers seguros con valores por defecto
- Optimizar Context para evitar re-renders innecesarios
- Combinar múltiples contextos efectivamente

Contenido

1. ¿Qué es Context API?

Context proporciona una forma de pasar datos a través del árbol de componentes sin tener que pasar props manualmente en cada nivel (prop drilling).

 Diagrama de Context

1.1 El Problema del Prop Drilling

```
// ✗ Prop drilling - pasar datos a través de muchos niveles
const App = () => {
  const [user, setUser] = useState<User | null>(null);

  return (
    <Layout user={user}>
      <Sidebar user={user}>
        <UserInfo user={user} />
      </Sidebar>
    </Layout>
  );
}
```

```

    <Main user={user}>
      <Header user={user} />
      <Content user={user}>
        <UserGreeting user={user} /> {/* 5 niveles de props! */}
      </Content>
    </Main>
  </Layout>
);
};

// ✅ Con Context - acceso directo al dato
const App = () => {
  return (
    <UserProvider>
      <Layout>
        <Sidebar>
          <UserInfo /> {/* Accede a user vía useUser() */}
        </Sidebar>
        <Main>
          <Header />
          <Content>
            <UserGreeting /> {/* Accede a user vía useUser() */}
          </Content>
        </Main>
      </Layout>
    </UserProvider>
  );
};

```

2. Crear Context con TypeScript

2.1 Patrón Básico

```

import { createContext, useContext, useState, ReactNode } from 'react';

// =====
// 1. DEFINIR TIPOS
// =====

// QUÉ: Interface que describe la estructura de un usuario
// PARA: Tipar los datos que el contexto compartirá
// IMPACTO: Autocompletado y errores en tiempo de compilación
interface User {
  id: string;
  name: string;
  email: string;
  role: 'admin' | 'user';
}

// QUÉ: Interface que define TODO lo que el contexto expone
// PARA: Documentar el contrato entre Provider y Consumers
// IMPACTO: Cualquier componente sabe exactamente qué puede usar
interface AuthContextValue {
  user: User | null;
  isAuthenticated: boolean;
}

```

```

    login: (email: string, password: string) => Promise<void>;
    logout: () => void;
    isLoading: boolean;
  }

  // =====
  // 2. CREAR CONTEXTO
  // =====

  // QUÉ: createContext con tipo union que incluye undefined
  // PARA: Forzar que el hook verifique si existe el Provider
  // IMPACTO: Previene el error de usar Context fuera de su Provider
  const AuthContext = createContext<AuthContextValue | undefined>(undefined);

  // Opción B: Con valor por defecto (no recomendado para datos dinámicos)
  // const AuthContext = createContext<AuthContextValue>({
  //   user: null,
  //   isAuthenticated: false,
  //   login: async () => {},
  //   logout: () => {},
  //   isLoading: false,
  // });

  // =====
  // 3. CREAR HOOK PERSONALIZADO
  // =====

  // QUÉ: Hook que encapsula useContext con validación
  // PARA: Garantizar que el contexto existe antes de usarlo
  // IMPACTO: Error claro si se usa fuera del Provider, tipo seguro dentro
  export const useAuth = (): AuthContextValue => {
    const context = useContext(AuthContext);

    if (context === undefined) {
      throw new Error('useAuth must be used within an AuthProvider');
    }

    return context;
  };

  // =====
  // 4. CREAR PROVIDER
  // =====

  interface AuthProviderProps {
    children: ReactNode;
  }

  // QUÉ: Componente Provider que mantiene el estado y lo comparte
  // PARA: Encapsular toda la lógica de autenticación en un solo lugar
  // IMPACTO: Cualquier descendiente puede acceder al estado sin prop drilling
  export const AuthProvider: React.FC<AuthProviderProps> = ({ children }) => {
    const [user, setUser] = useState<User | null>(null);
    const [isLoading, setIsLoading] = useState(false);

    const login = async (email: string, password: string) => {

```

```

    setIsLoading(true);
    try {
      // Simular llamada a API
      const response = await fetch('/api/login', {
        method: 'POST',
        body: JSON.stringify({ email, password }),
      });
      const userData = await response.json();
      setUser(userData);
    } finally {
      setIsLoading(false);
    }
  };

  const logout = () => {
    setUser(null);
  };

  const value: AuthContextValue = {
    user,
    isAuthenticated: user !== null,
    login,
    logout,
    isLoading,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

// =====
// 5. USO EN COMPONENTES
// =====

// En App.tsx
const App: React.FC = () => (
  <AuthProvider>
    <Router>
      <Routes />
    </Router>
  </AuthProvider>
);

// En cualquier componente descendiente
const Navbar: React.FC = () => {
  const { user, isAuthenticated, logout } = useAuth();

  return (
    <nav>
      {isAuthenticated ? (
        <>
          <span>Hola, {user?.name}</span>
          <button onClick={logout}>Cerrar sesión</button>
        </>
      ) : (
        <Link to="/login">Iniciar sesión</Link>
      )}
    </nav>
  );
};

```

```

    </nav>
  );
};

const ProtectedRoute: React.FC<{ children: ReactNode }> = ({ children }) => {
  const { isAuthenticated, isLoading } = useAuth();

  if (isLoading) {
    return <div>Cargando...</div>;
  }

  if (!isAuthenticated) {
    return <Navigate to="/login" />;
  }

  return <>{children}</>;
};

```

3. Context con Reducer

Para estados más complejos, combina Context con `useReducer` :

```

import {
  createContext,
  useContext,
  useReducer,
  ReactNode,
  Dispatch,
} from 'react';

// =====
// TIPOS
// =====

interface CartItem {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

interface CartState {
  items: CartItem[];
  total: number;
  itemCount: number;
}

// Acciones tipadas con discriminated unions
type CartAction =
  | { type: 'ADD_ITEM'; payload: Omit<CartItem, 'quantity'> }
  | { type: 'REMOVE_ITEM'; payload: { id: string } }
  | { type: 'UPDATE_QUANTITY'; payload: { id: string; quantity: number } }
  | { type: 'CLEAR_CART' };

interface CartContextValue {

```

```

state: CartState;
dispatch: Dispatch<CartAction>;
// Helpers para facilitar el uso
addItem: (item: Omit<CartItem, 'quantity'>) => void;
removeItem: (id: string) => void;
updateQuantity: (id: string, quantity: number) => void;
clearCart: () => void;
}

// =====
// REDUCER
// =====

const calculateTotals = (items: CartItem[]) => ({
  total: items.reduce((sum, item) => sum + item.price * item.quantity, 0),
  itemCount: items.reduce((sum, item) => sum + item.quantity, 0),
});

const cartReducer = (state: CartState, action: CartAction): CartState => {
  switch (action.type) {
    case 'ADD_ITEM': {
      const existingItem = state.items.find((i) => i.id === action.payload.id);

      let newItems: CartItem[];
      if (existingItem) {
        newItems = state.items.map((item) =>
          item.id === action.payload.id
            ? { ...item, quantity: item.quantity + 1 }
            : item,
        );
      } else {
        newItems = [...state.items, { ...action.payload, quantity: 1 }];
      }

      return { ...state, items: newItems, ...calculateTotals(newItems) };
    }

    case 'REMOVE_ITEM': {
      const newItems = state.items.filter((i) => i.id !== action.payload.id);
      return { ...state, items: newItems, ...calculateTotals(newItems) };
    }

    case 'UPDATE_QUANTITY': {
      if (action.payload.quantity <= 0) {
        const newItems = state.items.filter((i) => i.id !== action.payload.id);
        return { ...state, items: newItems, ...calculateTotals(newItems) };
      }

      const newItems = state.items.map((item) =>
        item.id === action.payload.id
          ? { ...item, quantity: action.payload.quantity }
          : item,
      );
      return { ...state, items: newItems, ...calculateTotals(newItems) };
    }
  }
}

```

```

    case 'CLEAR_CART':
      return { items: [], total: 0, itemCount: 0 };

    default:
      return state;
  }
};

// =====
// CONTEXTO Y PROVIDER
// =====

const CartContext = createContext<CartContextValue | undefined>(undefined);

export const useCart = () => {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error('useCart must be used within a CartProvider');
  }
  return context;
};

const initialState: CartState = {
  items: [],
  total: 0,
  itemCount: 0,
};

export const CartProvider: React.FC<{ children: ReactNode }> = ({
  children,
}) => {
  const [state, dispatch] = useReducer(cartReducer, initialState);

  // Helpers para simplificar el uso
  const addItem = (item: Omit<CartItem, 'quantity'>) => {
    dispatch({ type: 'ADD_ITEM', payload: item });
  };

  const removeItem = (id: string) => {
    dispatch({ type: 'REMOVE_ITEM', payload: { id } });
  };

  const updateQuantity = (id: string, quantity: number) => {
    dispatch({ type: 'UPDATE_QUANTITY', payload: { id, quantity } });
  };

  const clearCart = () => {
    dispatch({ type: 'CLEAR_CART' });
  };

  return (
    <CartContext.Provider
      value={{
        state,
        dispatch,
        addItem,

```

```

        removeItem,
        updateQuantity,
        clearCart,
      }}>
      {children}
    </CartContext.Provider>
  );
};

// =====
// USO
// =====

const ProductCard: React.FC<{ product: Product }> = ({ product }) => {
  const { addItem } = useCart();

  return (
    <div className="product-card">
      <h3>{product.name}</h3>
      <p>${product.price}</p>
      <button
        onClick={() =>
          addItem({ id: product.id, name: product.name, price: product.price })
        }>
        Agregar al carrito
      </button>
    </div>
  );
};

const CartSummary: React.FC = () => {
  const { state, clearCart } = useCart();

  return (
    <div className="cart-summary">
      <h3>Carrito ({state.itemCount} items)</h3>
      <ul>
        {state.items.map((item) => (
          <li key={item.id}>
            {item.name} x {item.quantity} = ${item.price * item.quantity}
          </li>
        ))}
      </ul>
      <p>
        <strong>Total: ${state.total.toFixed(2)}</strong>
      </p>
      <button onClick={clearCart}>Vaciar carrito</button>
    </div>
  );
};

```

4. Optimización de Context

4.1 El Problema de Re-renders

```
// ❌ PROBLEMA: Todos los consumidores se re-renderizan cuando cualquier valor cambia
const AppContext = createContext<
  | {
    user: User | null;
    theme: 'light' | 'dark';
    notifications: Notification[];
    setUser: (user: User) => void;
    setTheme: (theme: 'light' | 'dark') => void;
    addNotification: (n: Notification) => void;
  }
  | undefined
>(undefined);

// Si cambia 'notifications', componentes que solo usan 'theme' también se re-renderizan
```

4.2 Solución: Separar Contextos

```
// ✅ SOLUCIÓN: Separar por dominio/frecuencia de cambio

// Contexto de autenticación (cambia poco)
const AuthContext = createContext<AuthContextValue | undefined>(undefined);

// Contexto de tema (cambia poco)
const ThemeContext = createContext<ThemeContextValue | undefined>(undefined);

// Contexto de notificaciones (cambia frecuentemente)
const NotificationContext = createContext<NotificationContextValue | undefined>(
  undefined,
);

// Componente que solo necesita theme no se re-renderiza
// cuando cambian las notificaciones
const ThemeToggle: React.FC = () => {
  const { theme, setTheme } = useTheme();
  return (
    <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      {theme === 'light' ? '☀️' : '🌙'}
    </button>
  );
};
```

4.3 Separar Estado y Dispatch

```
// Para contextos con reducer, separar estado de dispatch

interface CounterState {
  count: number;
}

type CounterAction = { type: 'INCREMENT' } | { type: 'DECREMENT' };

// Contexto para el estado (cambia)
const CounterStateContext = createContext<CounterState | undefined>(undefined);
```

```

// Contexto para dispatch (nunca cambia)
const CounterDispatchContext = createContext<
  Dispatch<CounterAction> | undefined
>(undefined);

// Hooks separados
export const useCounterState = () => {
  const context = useContext(CounterStateContext);
  if (!context)
    throw new Error('useCounterState must be within CounterProvider');
  return context;
};

export const useCounterDispatch = () => {
  const context = useContext(CounterDispatchContext);
  if (!context)
    throw new Error('useCounterDispatch must be within CounterProvider');
  return context;
};

// Provider
export const CounterProvider: React.FC<{ children: ReactNode }> = ({
  children,
}) => {
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });

  return (
    <CounterStateContext.Provider value={state}>
      <CounterDispatchContext.Provider value={dispatch}>
        {children}
      </CounterDispatchContext.Provider>
    </CounterStateContext.Provider>
  );
};

// Componente que solo dispara acciones NO se re-renderiza cuando cambia el estado
const IncrementButton: React.FC = () => {
  const dispatch = useCounterDispatch();

  return <button onClick={() => dispatch({ type: 'INCREMENT' })}></button>;
};

// Componente que solo lee el estado se re-renderiza cuando cambia
const CountDisplay: React.FC = () => {
  const { count } = useCounterState();

  return <span>Count: {count}</span>;
};

```

4.4 Memoización del Value

```

import { useMemo, useCallback } from 'react';

const ThemeProvider: React.FC<{ children: ReactNode }> = ({ children }) => {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');

```

```
// Memoizar funciones para que no cambien en cada render
const toggleTheme = useCallback(() => {
  setTheme((prev) => (prev === 'light' ? 'dark' : 'light'));
}, []);

// Memoizar el value completo
const value = useMemo(
  () => ({
    theme,
    toggleTheme,
    isDark: theme === 'dark',
  }),
  [theme, toggleTheme],
);

return (
  <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>
);
};
```

5. Composición de Múltiples Providers

5.1 Provider Hell

```
// ❌ Difícil de leer y mantener
const App = () => (
  <AuthProvider>
    <ThemeProvider>
      <CartProvider>
        <NotificationProvider>
          <ModalProvider>
            <ToastProvider>
              <AppContent />
            </ToastProvider>
          </ModalProvider>
        </NotificationProvider>
      </CartProvider>
    </ThemeProvider>
  </AuthProvider>
);
```

5.2 Solución: Compose Providers

```
// Utility para componer providers
type Provider = React.FC<{ children: ReactNode }>;

const composeProviders = (...providers: Provider[]): Provider => {
  return ({ children }) =>
    providers.reduceRight(
      (acc, Provider) => <Provider>{acc}</Provider>,
      children,
    );
};
```

```
// Crear provider compuesto
const AppProviders = composeProviders(
  AuthProvider,
  ThemeProvider,
  CartProvider,
  NotificationProvider,
  ModalProvider,
  ToastProvider,
);

//  Uso limpio
const App = () => (
  <AppProviders>
    <AppContent />
  </AppProviders>
);
```

5.3 Alternativa: Provider Component

```
// providers/AppProviders.tsx
export const AppProviders: React.FC<{ children: ReactNode }> = ({
  children,
}) => (
  <AuthProvider>
    <ThemeProvider>
      <CartProvider>
        <NotificationProvider>{children}</NotificationProvider>
      </CartProvider>
    </ThemeProvider>
  </AuthProvider>
);

// App.tsx
const App = () => (
  <AppProviders>
    <AppContent />
  </AppProviders>
);
```

6. Context con Valores Iniciales desde Props

```
interface ThemeProviderProps {
  children: ReactNode;
  initialTheme?: 'light' | 'dark';
  storageKey?: string;
}

export const ThemeProvider: React.FC<ThemeProviderProps> = ({
  children,
  initialTheme = 'light',
  storageKey = 'app-theme',
}) => {
  // Intentar cargar tema de localStorage
```

```

const [theme, setTheme] = useState<'light' | 'dark'>(() => {
  if (typeof window === 'undefined') return initialTheme;

  const stored = localStorage.getItem(storageKey);
  return (stored as 'light' | 'dark') || initialTheme;
});

// Persistir cambios
useEffect(() => {
  localStorage.setItem(storageKey, theme);
  document.documentElement.setAttribute('data-theme', theme);
}, [theme, storageKey]);

const value = useMemo(
  () => ({
    theme,
    setTheme,
    toggleTheme: () => setTheme((t) => (t === 'light' ? 'dark' : 'light')),
  }),
  [theme],
);

return (
  <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>
);
};

// Uso con configuración
<ThemeProvider
  initialTheme="dark"
  storageKey="my-app-theme">
  <App />
</ThemeProvider>;

```

7. Cuándo Usar Context

Usar Context 	No Usar Context 
Datos globales (user, theme)	Estado local de un componente
Datos necesarios en muchos niveles	Datos solo 1-2 niveles abajo
Configuración de la app	Estado que cambia muy frecuentemente
Autenticación	Datos de formularios
Preferencias del usuario	Estados de UI específicos

8. Testing de Context

```

// Crear wrapper para tests
const createTestWrapper = (initialUser?: User) => {
  const wrapper: React.FC<{ children: ReactNode }> = ({ children }) => (
    <AuthProvider initialUser={initialUser}>{children}</AuthProvider>
  );
};

```

```
    return wrapper;
  };

// Test
describe('useAuth', () => {
  it('returns user when authenticated', () => {
    const mockUser = {
      id: '1',
      name: 'Test',
      email: 'test@test.com',
      role: 'user',
    };

    const { result } = renderHook(() => useAuth(), {
      wrapper: createTestWrapper(mockUser),
    });

    expect(result.current.user).toEqual(mockUser);
    expect(result.current.isAuthenticated).toBe(true);
  });

  it('throws error when used outside provider', () => {
    expect(() => {
      renderHook(() => useAuth());
    }).toThrow('useAuth must be used within an AuthProvider');
  });
});
```

✓ Checklist de Verificación

- ☐ Sé cuándo usar Context vs prop drilling
- ☐ Puedo crear contextos tipados con TypeScript
- ☐ Implemento hooks seguros que validan el contexto
- ☐ Entiendo cómo combinar Context con useReducer
- ☐ Sé optimizar Context para evitar re-renders
- ☐ Puedo componer múltiples providers limpiamente
- ☐ Entiendo cómo testear contextos

Recursos Adicionales

- [React Docs - Context](#)
- [React Docs - Scaling with Reducer + Context](#)
- [Kent C. Dodds - How to Use Context Effectively](#)

Fin del material teórico de Week-05

Ejercicio 01: Children Básico

Objetivo

Dominar el uso del prop `children` con tipado TypeScript para crear componentes contenedores flexibles.

Conceptos Clave

- Prop `children` como mecanismo de composición
- Tipado con `ReactNode` y `PropsWithChildren`
- Patrones de componentes contenedores
- Renderizado condicional de `children`

Duración Estimada

45 minutos

Instrucciones

Paso 1: Componente Card Básico

Aprenderás a crear un componente `Card` que acepta cualquier contenido como `children`.

Abre `starter/Card.tsx` y descomenta la sección correspondiente.

```
// QUÉ: Interface que define props con children tipado
// PARA: Permitir que Card acepte cualquier contenido React válido
// IMPACTO: TypeScript valida que children sea del tipo correcto
interface CardProps {
  children: ReactNode;
  className?: string;
}
```

Paso 2: Componente Alert con Variantes

Crearás un componente `Alert` que muestra `children` con diferentes estilos según la variante.

Abre `starter/Alert.tsx` y descomenta la sección correspondiente.

```
// QUÉ: Union type para variantes predefinidas
// PARA: Limitar las opciones de estilo a valores conocidos
// IMPACTO: IDE sugiere opciones válidas, errores en tiempo de compilación
type AlertVariant = 'info' | 'success' | 'warning' | 'error';
```

Paso 3: Componente Container Responsivo

Implementarás un `Container` que envuelve `children` con estilos de layout responsivo.

Abre `starter/Container.tsx` y descomenta la sección correspondiente.

```
// QUÉ: PropsWithChildren añade children?: ReactNode automáticamente
// PARA: Reducir código repetitivo al definir interfaces con children
// IMPACTO: Menos boilerplate, children siempre opcional
type ContainerProps = PropsWithChildren<{
  maxWidth?: 'sm' | 'md' | 'lg' | 'xl';
  centered?: boolean;
}>;
```

Paso 4: Componente Conditional Wrapper

Crearás un componente que condicionalmente envuelve sus `children`.

Abre `starter/ConditionalWrapper.tsx` y descomenta la sección correspondiente.

```
// QUÉ: Props para renderizado condicional de wrapper
// PARA: Envolver children solo si se cumple una condición
// IMPACTO: Evita duplicación de código y JSX anidado innecesario
interface ConditionalWrapperProps {
  condition: boolean;
  wrapper: (children: ReactNode) => ReactElement;
  children: ReactNode;
}
```

Paso 5: App de Demostración

Combina todos los componentes creados en una aplicación de demostración.

Abre `starter/App.tsx` y descomenta las secciones para ver los componentes en acción.

✓ Criterios de Evaluación

Criterio	Puntos
Card implementado con tipado correcto	5
Alert con variantes funcionales	5
Container con PropsWithChildren	5
ConditionalWrapper funcionando	3
Código limpio y bien comentado	2
Total	20

📁 Estructura del Ejercicio

```
ejercicio-01-children-basico/
├── README.md
├── starter/
│   ├── Card.tsx
│   ├── Alert.tsx
│   ├── Container.tsx
│   ├── ConditionalWrapper.tsx
│   └── App.tsx
└── solution/
    ├── Card.tsx
    ├── Alert.tsx
    ├── Container.tsx
    ├── ConditionalWrapper.tsx
    └── App.tsx
```

🔗 Recursos

- [React TypeScript Cheatsheet - Children](#)
- [React Docs - Passing Children](#)

Ejercicio 02: Slots Pattern

Objetivo

Implementar el patrón de slots para crear componentes con múltiples áreas de contenido flexible y tipadas.

Conceptos Clave

- Patrón Slots vs children único
- Props tipadas para cada slot
- Renderizado condicional de slots
- Composición avanzada con múltiples áreas

Duración Estimada

50 minutos

Instrucciones

Paso 1: Modal con Slots

Crearás un componente `Modal` con slots para header, body y footer.

Abre `starter/Modal.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Interface que define múltiples slots tipados
// PARA: Estructurar el modal con áreas predefinidas
// IMPACTO: API clara y predecible para el consumidor
interface ModalSlots {
  header?: ReactNode;
  footer?: ReactNode;
  children: ReactNode; // slot principal (body)
}
```

Paso 2: Layout con Slots

Implementarás un componente `PageLayout` con slots para sidebar, header y contenido principal.

Abre `starter/PageLayout.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Slots para layout de página completa
// PARA: Estructura consistente de página con áreas flexibles
// IMPACTO: Layouts predecibles sin sacrificar flexibilidad
interface PageLayoutSlots {
  header?: ReactNode;
  sidebar?: ReactNode;
  children: ReactNode; // contenido principal
  footer?: ReactNode;
}
```

Paso 3: Card Avanzado con Slots

Crearás un `AdvancedCard` con slots para imagen, acciones y badges.

Abre `starter/AdvancedCard.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Card con múltiples áreas de contenido
// PARA: Tarjetas ricas en contenido manteniendo flexibilidad
// IMPACTO: Componente versátil para diferentes casos de uso
interface AdvancedCardSlots {
  image?: ReactNode;
  badge?: ReactNode;
  actions?: ReactNode;
  children: ReactNode;
  title: string;
}
```

Paso 4: App de Demostración

Integra todos los componentes con slots en una aplicación completa.

Abre `starter/App.tsx` y descomenta las secciones para ver todo en acción.

✓ Criterios de Evaluación

Criterio	Puntos
Modal con slots correctamente tipados	6
PageLayout funcional con todos los slots	6
AdvancedCard con slots flexibles	5
Código limpio y bien documentado	3
Total	20

📁 Estructura del Ejercicio

```
ejercicio-02-slots-pattern/
├─ README.md
├─ starter/
│   ├── Modal.tsx
│   ├── PageLayout.tsx
│   ├── AdvancedCard.tsx
│   └─ App.tsx
└─ solution/
    ├── Modal.tsx
    ├── PageLayout.tsx
    ├── AdvancedCard.tsx
    └─ App.tsx
```

🔗 Recursos

- [React Patterns - Slots](#)
- [Kent C. Dodds - Compound Components](#)

Ejercicio 03: Compound Components

Objetivo

Implementar el patrón Compound Components para crear APIs de componentes declarativas y flexibles utilizando Context interno.

Conceptos Clave

- Patrón Compound Components
- Context interno (no exportado)
- Sub-componentes adjuntos al componente principal
- Estado compartido implícito

Duración Estimada

60 minutos

Instrucciones

Paso 1: Accordion Compound Component

Crearás un componente `Accordion` con sub-componentes `Item`, `Trigger` y `Content`.

Abre `starter/Accordion.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Context interno para compartir estado del accordion
// PARA: Comunicar estado entre subcomponentes sin prop drilling
// IMPACTO: API limpia y declarativa para el usuario
interface AccordionContextValue {
  openItems: Set<string>;
  toggleItem: (id: string) => void;
  allowMultiple: boolean;
}
```

Paso 2: Tabs Compound Component

Implementarás un componente `Tabs` con sub-componentes `List`, `Tab` y `Panel`.

Abre `starter/Tabs.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Context para sistema de tabs
// PARA: Coordinar estado activo entre Tab y Panel
// IMPACTO: Tabs sincronizadas automáticamente
interface TabsContextValue {
  activeTab: string;
  setActiveTab: (id: string) => void;
}
```

Paso 3: Select Compound Component

Crearás un componente `Select` con sub-componentes `Trigger`, `Content` y `Option`.

Abre `starter/Select.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Context para select/dropdown personalizado
// PARA: Estado de apertura y selección compartido
// IMPACTO: Select flexible sin dependencias externas
interface SelectContextValue {
```

```
isOpen: boolean;
toggle: () => void;
value: string;
onChange: (value: string) => void;
}
```

Paso 4: App de Demostración

Integra todos los compound components en una aplicación.

Abre `starter/App.tsx` y descomenta las secciones.

✓ Criterios de Evaluación

Criterio	Puntos
Accordion con Context interno funcional	8
Tabs con sincronización correcta	7
Select con estado compartido	7
Código bien tipado y documentado	3
Total	25

📁 Estructura del Ejercicio

```
ejercicio-03-compound-components/
├── README.md
├── starter/
│   ├── Accordion.tsx
│   ├── Tabs.tsx
│   ├── Select.tsx
│   └── App.tsx
└── solution/
    ├── Accordion.tsx
    ├── Tabs.tsx
    ├── Select.tsx
    └── App.tsx
```

🔗 Recursos

- [Kent C. Dodds - Compound Components](#)
- [React Patterns - Compound Components](#)

Ejercicio 04: Context Básico

🎯 Objetivo

Implementar Context API con TypeScript para compartir estado global de forma segura y tipada.

📖 Conceptos Clave

- `createContext` con TypeScript

- Custom hooks para consumir Context
- Provider pattern
- Tipado seguro con undefined



Duración Estimada

45 minutos



Instrucciones

Paso 1: AuthContext

Crearás un contexto de autenticación con usuario y funciones de login/logout.

Abre `starter/AuthContext.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: createContext con undefined inicial fuerza validación
// PARA: Asegurar que el hook se use dentro del Provider
// IMPACTO: Errores claros en desarrollo, no en producción
const AuthContext = createContext<AuthContextValue | undefined>(undefined);
```

Paso 2: ThemeContext

Implementarás un contexto de tema (light/dark) con persistencia.

Abre `starter/ThemeContext.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Contexto para preferencia de tema
// PARA: Aplicar tema globalmente en la aplicación
// IMPACTO: Cambio de tema sin prop drilling
interface ThemeContextValue {
  theme: 'light' | 'dark';
  toggleTheme: () => void;
}
```

Paso 3: LanguageContext

Crearás un contexto de idioma para internacionalización básica.

Abre `starter/LanguageContext.tsx` y descomenta las secciones correspondientes.

Paso 4: App de Demostración

Integra todos los contextos y demuestra su uso.

Abre `starter/App.tsx` y descomenta las secciones.



Criterios de Evaluación

Criterio	Puntos
AuthContext con hook personalizado	7
ThemeContext con persistencia	6
LanguageContext funcional	5
Código bien tipado y documentado	2

Total	20
-------	----

Estructura del Ejercicio

```
ejercicio-04-context-basico/  
├── README.md  
├── starter/  
│   ├── AuthContext.tsx  
│   ├── ThemeContext.tsx  
│   ├── LanguageContext.tsx  
│   └── App.tsx  
└── solution/  
    ├── AuthContext.tsx  
    ├── ThemeContext.tsx  
    ├── LanguageContext.tsx  
    └── App.tsx
```

Recursos

- [React Docs - useContext](#)
- [React TypeScript Cheatsheet - Context](#)

Ejercicio 05: Context Avanzado con useReducer

Objetivo

Implementar patrones avanzados de Context API combinando `useReducer` para manejo de estado complejo y optimización de rendimiento.

Conceptos Clave

- `useReducer` con TypeScript
- Actions tipadas con discriminated unions
- Separación de State y Dispatch contexts
- Context con acciones asíncronas
- Optimización con memo y context splitting

Duración Estimada

60 minutos

Instrucciones

Paso 1: CartContext con useReducer

Crearás un carrito de compras con estado complejo manejado por reducer.

Abre `starter/CartContext.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: useReducer maneja transiciones de estado complejas  
// PARA: Estado predecible con acciones tipadas  
// IMPACTO: Debugging más fácil, código más mantenible  
type CartAction =
```

```
| { type: 'ADD_ITEM'; payload: Product }  
| { type: 'REMOVE_ITEM'; payload: string }  
| { type: 'UPDATE_QUANTITY'; payload: { id: string; quantity: number } }  
| { type: 'CLEAR_CART' };
```

Paso 2: NotificationContext con Queue

Implementarás un sistema de notificaciones con cola y auto-dismiss.

Abre `starter/NotificationContext.tsx` y descomenta las secciones correspondientes.

Paso 3: Context Splitting para Optimización

Separar State y Dispatch en contextos diferentes para evitar re-renders innecesarios.

Abre `starter/OptimizedContext.tsx` y descomenta las secciones correspondientes.

```
// QUÉ: Separar State y Dispatch en contextos diferentes  
// PARA: Evitar re-renders en componentes que solo dispatchean  
// IMPACTO: Mejor rendimiento en apps grandes
```

Paso 4: App de Demostración

Integra todos los contextos avanzados.

Abre `starter/App.tsx` y descomenta las secciones.

✓ Criterios de Evaluación

Criterio	Puntos
CartContext con useReducer completo	8
NotificationContext con queue	6
Context splitting implementado	4
Código bien tipado y documentado	2
Total	20

📁 Estructura del Ejercicio

```
ejercicio-05-context-avanzado/  
├─ README.md  
├─ starter/  
│   ├─ CartContext.tsx  
│   ├─ NotificationContext.tsx  
│   ├─ OptimizedContext.tsx  
│   └─ App.tsx  
└─ solution/  
    ├─ CartContext.tsx  
    ├─ NotificationContext.tsx  
    ├─ OptimizedContext.tsx  
    └─ App.tsx
```

Recursos

- [React Docs - useReducer](#)
- [TypeScript Discriminated Unions](#)

Proyecto Semana 05: Sistema de Configuración UI

Objetivo

Construir un sistema de configuración de interfaz de usuario utilizando composición de componentes, Context API y compound components para crear una experiencia de usuario rica y configurable.

Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio]

Adapta el proyecto al dominio asignado. Ejemplos:

- **Biblioteca:** Configurador de vista de catálogo
- **Farmacia:** Panel de preferencias de inventario
- **Gimnasio:** Configurador de dashboard de entrenamiento
- **Restaurante:** Panel de configuración de menú digital

Descripción

Crearás una aplicación con un panel de configuración que permite al usuario personalizar:

1. **Tema visual** (claro/oscuro/sistema)
2. **Tamaño de texto** (pequeño/mediano/grande)
3. **Densidad de contenido** (compacto/normal/espacioso)
4. **Preferencias de notificaciones**

La aplicación debe demostrar:

- Context API para estado global de configuración
- Compound Components para formularios y cards
- Composición con children y slots
- Persistencia de preferencias en localStorage

Tiempo Estimado

2-2.5 horas

Requisitos Funcionales

1. ConfigProvider (Context)

```
interface ConfigState {
  theme: 'light' | 'dark' | 'system';
  fontSize: 'small' | 'medium' | 'large';
  density: 'compact' | 'normal' | 'spacious';
  notifications: {
    email: boolean;
    push: boolean;
    sound: boolean;
  };
}
```

```
};  
}
```

- Persistir en localStorage
- Detectar preferencia del sistema para tema
- Hook `useConfig` con validación

2. Compound Components

Card Compound Component:

```
<Card>  
  <Card.Header>  
    <Card.Title>Título</Card.Title>  
    <Card.Actions>...</Card.Actions>  
  </Card.Header>  
  <Card.Body>Contenido</Card.Body>  
  <Card.Footer>Pie</Card.Footer>  
</Card>
```

Form Compound Component:

```
<Form onSubmit={handleSubmit}>  
  <Form.Field>  
    <Form.Label>Email</Form.Label>  
    <Form.Input  
      type="email"  
      name="email"  
    />  
    <Form.Error>Error message</Form.Error>  
  </Form.Field>  
  <Form.Actions>  
    <Form.Submit>Guardar</Form.Submit>  
  </Form.Actions>  
</Form>
```

3. Panel de Configuración

- Sección de tema con preview en tiempo real
- Sección de tipografía con slider o botones
- Sección de densidad con preview
- Sección de notificaciones con toggles
- Botón de resetear a valores por defecto

4. Página Principal

- Mostrar datos del dominio asignado
- Cards que respetan la configuración
- Demostrar que los cambios se aplican globalmente

Estructura del Proyecto

```
3-proyecto/  
├─ README.md  
├─ starter/  
│   └─ contexts/
```

```

├── ConfigContext.tsx
├── components/
│   ├── Card/
│   │   └── Card.tsx
│   ├── Form/
│   │   └── Form.tsx
│   ├── SettingsPanel/
│   │   └── SettingsPanel.tsx
│   └── Layout/
│       └── Layout.tsx
├── hooks/
│   └── useLocalStorage.ts
├── App.tsx
└── solution/
    └── [misma estructura con implementación completa]

```

✓ Criterios de Evaluación

Criterio	Puntos
ConfigContext con persistencia	15
Card Compound Component completo	15
Form Compound Component funcional	15
Panel de configuración con todas las opciones	15
Aplicación de estilos según configuración	15
Código limpio, tipado y documentado	10
Adaptación coherente al dominio	15
Total	100

💡 Hints

1. useLocalStorage hook:

```

const useLocalStorage = <T>(key: string, initialValue: T) => {
  // Implementación que sincroniza con localStorage
};

```

2. CSS Variables para temas:

```

[data-theme='dark'] {
  --bg-primary: #1a1a2e;
  --text-primary: #ffffff;
}

```

3. Compound Components con Object.assign:

```

const Card = Object.assign(CardRoot, {
  Header: CardHeader,
  Title: CardTitle,
});

```

```
Body: CardBody,  
Footer: CardFooter,  
});
```

Recursos

- [React Docs - Context](#)
- [React Patterns - Compound Components](#)
- [CSS Variables](#)

Glosario - Semana 05

Composición de Componentes y Context API

A

API (Application Programming Interface)

Conjunto de funciones y procedimientos que permiten la comunicación entre componentes de software. En React, Context API es una interfaz para compartir datos entre componentes.

C

Children (Prop)

Propiedad especial en React que contiene los elementos JSX anidados dentro de un componente. Permite la composición de componentes.

```
interface CardProps {  
  children: ReactNode; // Acepta cualquier contenido válido de React  
}
```

Composition (Composición)

Patrón de diseño donde componentes complejos se construyen combinando componentes más simples. React favorece composición sobre herencia.

```
// Composición: Card contiene Header y Body  
<Card>  
  <Card.Header>Título</Card.Header>  
  <Card.Body>Contenido</Card.Body>  
</Card>
```

Compound Components

Patrón donde un componente padre y sus hijos trabajan juntos para lograr una funcionalidad, compartiendo estado implícitamente a través de Context.

```
// Compound Component: Tabs  
<Tabs>  
  <Tabs.List>  
    <Tabs.Tab>Tab 1</Tabs.Tab>  
  </Tabs.List>
```

```
<Tabs.Panel>Contenido 1</Tabs.Panel>
</Tabs>
```

Consumer

Componente que lee valores de un Context. En la API moderna, se usa el hook `useContext` en lugar del componente Consumer.

Context

Mecanismo de React para pasar datos a través del árbol de componentes sin necesidad de pasar props manualmente en cada nivel (evita prop drilling).

createContext

Función de React que crea un objeto Context con Provider y Consumer.

```
const ThemeContext = createContext<Theme | undefined>(undefined);
```

D

Discriminated Union

Patrón de TypeScript donde cada tipo en una union tiene una propiedad literal que lo identifica únicamente.

```
type Action = { type: 'INCREMENT' } | { type: 'SET'; payload: number };
```

Dispatch

Función que envía acciones al reducer para actualizar el estado.

```
const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'INCREMENT' });
```

H

HOC (Higher-Order Component)

Función que recibe un componente y retorna un nuevo componente con funcionalidad adicional. Patrón menos usado desde la introducción de hooks.

```
const withTheme = (Component) => (props) => {
  const theme = useTheme();
  return <Component {...props} theme={theme} />;
};
```

I

Implicit State Sharing

Compartir estado entre componentes sin pasarlo explícitamente via props, típicamente usando Context interno.

L

localStorage

API del navegador para almacenar datos persistentes en el cliente. Útil para guardar preferencias de usuario.

```
localStorage.setItem('theme', 'dark');  
const theme = localStorage.getItem('theme');
```

P

Prop Drilling

Anti-patrón donde props se pasan a través de múltiples niveles de componentes que no los necesitan, solo para llegar a componentes más profundos.

```
// ❌ Prop Drilling  
<App user={user}>  
  <Layout user={user}>  
    <Sidebar user={user}>  
      <UserProfile user={user} /> {/* Único que lo usa */}  
    </Sidebar>  
  </Layout>  
</App>
```

Provider

Componente que hace disponible un valor de Context a todos sus descendientes.

```
<ThemeContext.Provider value={theme}>{children}</ThemeContext.Provider>
```

PropsWithChildren

Tipo de TypeScript que agrega `children: ReactNode` a un tipo de props.

```
type ButtonProps = PropsWithChildren<{  
  onClick: () => void;  
}>;
```

R

ReactElement

Tipo que representa un elemento React creado con JSX. Más específico que `ReactNode`.

```
interface SlotProps {  
  header: ReactElement; // Solo elementos, no strings  
}
```

ReactNode

Tipo que representa cualquier cosa que React puede renderizar: elementos, strings, números, null, undefined, arrays, fragments.

```
interface CardProps {  
  children: ReactNode; // Acepta cualquier contenido  
}
```

Reducer

Función pura que calcula el nuevo estado basándose en el estado actual y una acción.

```
const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    default:
      return state;
  }
};
```

Render Props

Patrón donde un componente recibe una función como prop que retorna elementos React. Permite compartir lógica entre componentes.

```
<Mouse render={({mouse}) => <Cat position={mouse} />} />
```

S

Slots Pattern

Patrón de composición donde un componente define "ranuras" (slots) para contenido específico, permitiendo personalización flexible.

```
interface LayoutProps {
  header: ReactNode;
  sidebar: ReactNode;
  content: ReactNode;
}
```

State Colocation

Principio de mantener el estado lo más cerca posible de donde se usa. Solo elevar a Context cuando múltiples componentes lo necesitan.

U

useContext

Hook de React que permite leer valores de un Context.

```
const theme = useContext(ThemeContext);
```

useReducer

Hook de React para manejar estado complejo con un reducer. Alternativa a useState para lógica de estado más elaborada.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

V

Value Prop

Propiedad del Provider que especifica el valor que estará disponible para todos los consumidores del Context.

```
<Context.Provider value={{ user, login, logout }}>
```

Referencias Rápidas

Término	Uso Principal
children	Composición básica
createContext	Crear contexto
useContext	Consumir contexto
useReducer	Estado complejo
Provider	Proveer valores
ReactNode	Tipo para children
Compound	Componentes relacionados
Slots	Múltiples áreas de contenido

Ejemplo Integrado

```
// Compound Component con Context interno
const TabsContext = createContext<TabsContextValue | undefined>(undefined);

const Tabs = Object.assign(
  ({ children, defaultTab }: TabsProps) => {
    const [activeTab, setActiveTab] = useState(defaultTab);
    return (
      <TabsContext.Provider value={{ activeTab, setActiveTab }}>
        <div className="tabs">{children}</div>
      </TabsContext.Provider>
    );
  },
  {
    List: TabsList, // Slot para lista de tabs
    Tab: Tab, // Tab individual
    Panel: TabPanel, // Panel de contenido
  },
);

// Uso
<Tabs defaultTab="home">
  <Tabs.List>
    <Tabs.Tab id="home">Inicio</Tabs.Tab>
    <Tabs.Tab id="settings">Config</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel id="home">Contenido inicio</Tabs.Panel>
  <Tabs.Panel id="settings">Contenido config</Tabs.Panel>
</Tabs>;
```

Semana 05: Composición de Componentes y Context API

Objetivos de Aprendizaje

Al finalizar esta semana, serás capaz de:

- ✓ Aplicar patrones de composición para crear componentes reutilizables
- ✓ Usar children y render props para flexibilidad en componentes
- ✓ Implementar el patrón Compound Components
- ✓ Entender y aplicar Context API con TypeScript
- ✓ Crear providers tipados para compartir estado global
- ✓ Optimizar Context para evitar re-renders innecesarios
- ✓ Combinar múltiples contextos efectivamente

Requisitos Previos

- ✓ Week-01: Fundamentos de TypeScript
- ✓ Week-02: Introducción a React y JSX/TSX
- ✓ Week-03: Props, Estado y Eventos
- ✓ Week-04: Renderizado Condicional y Listas

Estructura de la Semana

```
week-05/  
├─ README.md                # Este archivo  
├─ rubrica-evaluacion.md    # Criterios de evaluación  
├─ 0-assets/                # Diagramas SVG  
├─ 1-teoria/                # Material teórico  
│   ├── 01-composicion-componentes.md  
│   ├── 02-children-y-slots.md  
│   ├── 03-compound-components.md  
│   └─ 04-context-api.md  
├─ 2-ejercicios/            # Ejercicios guiados  
│   ├── ejercicio-01-children-basico/  
│   ├── ejercicio-02-slots-pattern/  
│   ├── ejercicio-03-compound-components/  
│   ├── ejercicio-04-context-basico/  
│   └─ ejercicio-05-context-avanzado/  
├─ 3-proyecto/              # Proyecto semanal  
│   ├── README.md  
│   ├── starter/  
│   └─ solution/  
├─ 4-recursos/              # Recursos adicionales  
│   ├── ebooks-free/  
│   ├── videografia/  
│   └─ webgrafia/  
└─ 5-glosario/              # Términos clave  
    └─ README.md
```

Contenidos

1. Teoría (2-2.5 horas)

Archivo	Tema	Duración
01-composicion-componentes.md	Patrones de composición vs herencia	30 min

02-children-y-slots.md	Children, ReactNode, slots pattern	30 min
03-compound-components.md	Compound Components pattern	30 min
04-context-api.md	Context API con TypeScript	45 min

2. Ejercicios Guiados (3-3.5 horas)

Ejercicio	Tema	Duración
Ejercicio 01	Children básico y tipado	30 min
Ejercicio 02	Slots pattern con TypeScript	40 min
Ejercicio 03	Compound Components	45 min
Ejercicio 04	Context API básico	40 min
Ejercicio 05	Context con reducers	45 min

3. Proyecto Semanal (2-2.5 horas)

Sistema de Temas con Panel de Configuración

Crear una aplicación que use composición y Context para:

- Implementar tema claro/oscuro global
- Crear componentes compuestos (Card, Modal, Tabs)
- Gestionar preferencias de usuario con Context
- Aplicar patrones de composición avanzados



Distribución del Tiempo (8 horas)

Actividad	Tiempo
Teoría	2-2.5h
Ejercicios	3-3.5h
Proyecto	2-2.5h
Total	~8h



Entregable



Proyecto Semanal 📦 (100%)

Sistema de Temas con Panel de Configuración - Aplicación completa que demuestre dominio de composición de componentes y Context API.



Nota: La teoría y los ejercicios son recursos de aprendizaje para prepararte para el proyecto. No son entregables evaluados.



Checklist de la Semana

Preparación (No evaluada):

- ☐ Leer material teórico de composición y Context
- ☐ Completar ejercicios guiados para practicar

- ☐ Revisar ejemplos de compound components

Entregable (Evaluado):

- ☐ Proyecto semanal completado y funcional
- ☐ Implementación coherente con el dominio asignado
- ☐ Código TypeScript limpio y bien tipado
- ☐ Al menos 2 compound components implementados
- ☐ Sistema de tema claro/oscurο funcional

Navegación

← Anterior	Siguiente →
Week-04: Renderizado Condicional	Week-06: Custom Hooks

Recursos Rápidos

- [React Docs - Composition vs Inheritance](#)
- [React Docs - Context](#)
- [Patterns.dev - Compound Pattern](#)

Semana 05 de 20 · Etapa 2: Fundamentos de React

Rúbrica de Evaluación - Semana 05

Composición de Componentes y Context API

Competencias a Evaluar

Competencia	Descripción
Composición	Crear componentes flexibles usando children y slots
Compound Components	Implementar patrones de componentes compuestos
Context API	Usar Context para compartir estado global
TypeScript	Tipar correctamente providers y consumers

Distribución de la Evaluación

Evidencia	Peso	Descripción
Proyecto 	100%	Sistema de Temas con Panel de Configuración

 **Nota:** La teoría y los ejercicios son recursos de aprendizaje para prepararte para el proyecto. No son entregables evaluados.

Recursos de Aprendizaje (No Evaluados)

Teoría

Material de estudio para comprender los conceptos:

- Composición vs herencia en React
- Children y ReactNode tipados
- Patrones de compound components
- Context API con TypeScript

Ejercicios Guiados

Práctica para dominar las técnicas antes del proyecto:

Ejercicio	Objetivo de Aprendizaje
01 - Children Básico	Tipar children, ReactNode, ReactElement
02 - Slots Pattern	Named slots, composición flexible
03 - Compound Components	API compuesta, estado compartido interno
04 - Context Básico	createContext, Provider, useContext
05 - Context Avanzado	Reducer, separación, optimización

 Criterios de Evaluación del Proyecto (100%)

Distribución de Puntos

Aspecto	Peso	Descripción
Funcionalidad	40%	Features completas y funcionando correctamente
Adaptación al Dominio	35%	Implementación coherente con dominio asignado
Calidad del Código	25%	TypeScript, organización, buenas prácticas

Funcionalidad (40%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Theme Context	Tema global con persistencia	Tema funcional con toggle	Toggle básico	No implementa tema
Componentes compuestos	API intuitiva y flexible	Componentes bien estructurados	Estructura básica correcta	Componentes mal diseñados
Integración	Todos los componentes integrados	Mayoría integrados	Integración parcial	Sin integración

Adaptación al Dominio (35%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Coherencia	Perfecta adaptación al dominio asignado	Buena adaptación	Adaptación parcial	No adapta al dominio
Originalidad	Solución creativa y única	Implementación propia	Solución básica	Copia evidente

Contexto	Datos y UI coherentes con el negocio	Mayoría coherente	Parcialmente coherente	Incoherente
----------	--------------------------------------	-------------------	------------------------	-------------

Calidad del Código (25%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
TypeScript	Tipos estrictos sin any	Buenos tipos	Tipos básicos	Uso excesivo de any
Código limpio	Excelente organización y documentación	Código bien estructurado	Legible pero mejorable	Desorganizado
Patrones	Aplica patrones correctamente	Usa patrones básicos	Intenta usar patrones	No usa patrones

 Desglose del Proyecto (100%)

Componente	Puntos	Criterios
ThemeProvider	20	Context tipado, toggle funcional
Compound Card	20	Card.Header, Card.Body, Card.Footer
Compound Modal	20	Modal.Trigger, Modal.Content, Modal.Close
Compound Tabs	20	Tabs.List, Tabs.Tab, Tabs.Panel
Integración y Dominio	20	Composición de contextos + dominio único
Total	100	

 Lista de Verificación

 Preparación (No Evaluada)

- ☐ Leer material teórico de composición y Context API
- ☐ Completar ejercicios guiados para practicar técnicas
- ☐ Revisar ejemplos de compound components
- ☐ Entender flujo Provider → Consumer
- ☐ Practicar tipado de children y ReactNode

 Proyecto (Evaluado - 100%)

- ☐ Sistema de tema claro/oscuro funcional
- ☐ ThemeProvider con Context tipado
- ☐ Al menos 2 compound components implementados
- ☐ Código TypeScript sin errores (sin any)
- ☐ Componentes reutilizables y flexibles
- ☐ Implementación coherente con dominio asignado
- ☐ Documentación clara del uso

 Niveles de Logro

Nivel	Rango	Descripción
Experto	95-100%	Domina composición y Context, código ejemplar
Avanzado	85-94%	Implementa patrones correctamente con buen tipado
Competente	75-84%	Entiende y aplica conceptos principales
En desarrollo	70-74%	Necesita práctica adicional
Insuficiente	<70%	Requiere refuerzo de fundamentos

✓ Criterios de Aprobación

- **Mínimo 70%** en el proyecto para aprobar la semana
- El proyecto debe ser funcional y ejecutable
- Implementación coherente con el dominio asignado
- Código original (sin copias de otros aprendices)

📌 Notas para el Evaluador

1. **Composición:** Verificar que no se use herencia de clases
2. **Tipado de children:** Revisar uso de `ReactNode` vs tipos específicos
3. **Context:** Evaluar si el valor por defecto es seguro (no `undefined` sin manejo)
4. **Compound Components:** Validar que el estado se comparte internamente
5. **Dominio único:** Verificar implementación coherente con el dominio asignado
6. **Originalidad:** Comparar con entregas de otros aprendices

📁 Formato de Entrega

```
week-05-entrega/  
├── README.md           # Descripción del proyecto y dominio  
├── src/  
│   ├── contexts/      # ThemeContext y otros contextos  
│   ├── components/    # Compound components (Card, Modal, Tabs)  
│   └── App.tsx         # Integración de componentes  
├── package.json  
└── tsconfig.json
```

Entregable único: Proyecto comprimido o enlace a repositorio

🔗 Recursos de Apoyo

- [React Patterns](#)
- [TypeScript + React Cheatsheet](#)
- [Kent C. Dodds - Compound Components](#)

Rúbrica Week-05 · Composición de Componentes y Context API